

**Laxmi Charitable Trust's**  
**Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce**  
**Department of Information Technology (B.Sc.I.T Semester V)**  
**Applied Business Analytics Practical**  
**Practical – V**

|                             |                                  |
|-----------------------------|----------------------------------|
| Roll No.:T007               | Name:Subhashree Jena             |
| Class:TYIT                  | Batch:1                          |
| Date of Assignment:21/08/26 | Date/Time of Submission:21/08/26 |

**5. Probability and Probability Distributions:**

**a. Simulate a probability experiment (e.g., rolling dice) using programming and calculate the probabilities of different outcomes.**

**1. Simulate 1,000 dice rolls and display the first 20 outcomes.**

**Code:-**

```
import numpy as np
dice_rolls = np.random.randint(1, 7, 1000)
print("First 20 dice outcomes:")
print(dice_rolls[:20])
```

**Output:-**

```
First 20 dice outcomes:
[2 3 3 5 5 2 6 1 4 1 2 2 1 4 1 5 3 6 5 3]
```

**2. Calculate the experimental probability of getting each number from 1 to 6.**

**Code:-**

```
import numpy as np
dice_rolls = np.random.randint(1, 7, 1000)
print("Experimental Probability:")
for i in range(1, 7):
    count = np.sum(dice_rolls == i)
    probability = count / 1000
    print("P(", i, ") =", probability)
```

**Output:-**

```
Experimental Probability:
P( 1 ) = 0.178
P( 2 ) = 0.179
P( 3 ) = 0.152
P( 4 ) = 0.169
P( 5 ) = 0.154
P( 6 ) = 0.168
```

**3. Calculate the theoretical probability of getting each number from 1 to 6.**

**Code:-**

```
print("Theoretical Probability:")
for i in range(1, 7):
    probability = 1 / 6
    print("P(", i, ") =", probability)
```

**Output:-**

```
Theoretical Probability:  
P( 1 ) = 0.16666666666666666  
P( 2 ) = 0.16666666666666666  
P( 3 ) = 0.16666666666666666  
P( 4 ) = 0.16666666666666666  
P( 5 ) = 0.16666666666666666  
P( 6 ) = 0.16666666666666666
```

**4. Find the probability of getting 1, 2, 3, 4, 5, 6.****Code:-**

```
print("Probability of each number:")  
for i in range(1, 7):  
    print("P(", i, ") = 1/6 =", round(1/6, 4))
```

**Output:-**

```
Probability of each number:  
P( 1 ) = 1/6 = 0.1667  
P( 2 ) = 1/6 = 0.1667  
P( 3 ) = 1/6 = 0.1667  
P( 4 ) = 1/6 = 0.1667  
P( 5 ) = 1/6 = 0.1667  
P( 6 ) = 1/6 = 0.1667
```

**5. Find the probability of getting an even number.****Code:-**

```
import numpy as np  
dice_rolls = np.random.randint(1, 7, 1000)  
even_count = np.sum(dice_rolls % 2 == 0)  
probability = even_count / 1000  
print("Even numbers =", even_count)  
print("Experimental Probability of Even =", probability)  
print("Theoretical Probability of Even =", 3/6)
```

**Output:-**

```
Even numbers = 480  
Experimental Probability of Even = 0.48  
Theoretical Probability of Even = 0.5
```

**6. Find the probability of getting an odd number.****Code:-**

```
import numpy as np  
dice_rolls = np.random.randint(1, 7, 1000)  
odd_count = np.sum(dice_rolls % 2 != 0)  
probability = odd_count / 1000  
print("Odd numbers =", odd_count)  
print("Experimental Probability of Odd =", probability)  
print("Theoretical Probability of Odd =", 3/6)
```

**Output:-**

```
Odd numbers = 530  
Experimental Probability of Odd = 0.53  
Theoretical Probability of Odd = 0.5
```

### 7. Find the probability of getting a number greater than 4.

#### Code:-

```
import numpy as np
dice_rolls = np.random.randint(1, 7, 1000)
count = np.sum(dice_rolls > 4)
probability = count / 1000
print("Numbers greater than 4 =", count)
print("Experimental Probability =", probability)
print("Theoretical Probability =", 2/6)
```

#### Output:-

```
Numbers greater than 4 = 346
Experimental Probability = 0.346
Theoretical Probability = 0.3333333333333333
```

### 8. Find the probability of getting a number less than 4.

#### Code:-

```
import numpy as np
dice_rolls = np.random.randint(1, 7, 1000)
count = np.sum(dice_rolls < 4)
probability = count / 1000
print("Numbers less than 4 =", count)
print("Experimental Probability =", probability)
print("Theoretical Probability =", 3/6)
```

#### Output:-

```
Numbers less than 4 = 470
Experimental Probability = 0.47
Theoretical Probability = 0.5
```

### 9. Find the probability of getting a number greater than or equal to 4.

#### Code:-

```
import numpy as np
dice_rolls = np.random.randint(1, 7, 1000)
count = np.sum(dice_rolls >= 4)
probability = count / 1000
print("Numbers greater than or equal to 4 =", count)
print("Experimental Probability =", probability)
print("Theoretical Probability =", 3/6)
```

#### Output:-

```
Numbers less than 4 = 470
Experimental Probability = 0.47
Theoretical Probability = 0.5
```

### 10. Create a bar chart showing the frequency of outcomes 1 to 6.

#### Code:-

```
import numpy as np
import matplotlib.pyplot as plt
dice_rolls = np.random.randint(1, 7, 1000)
```

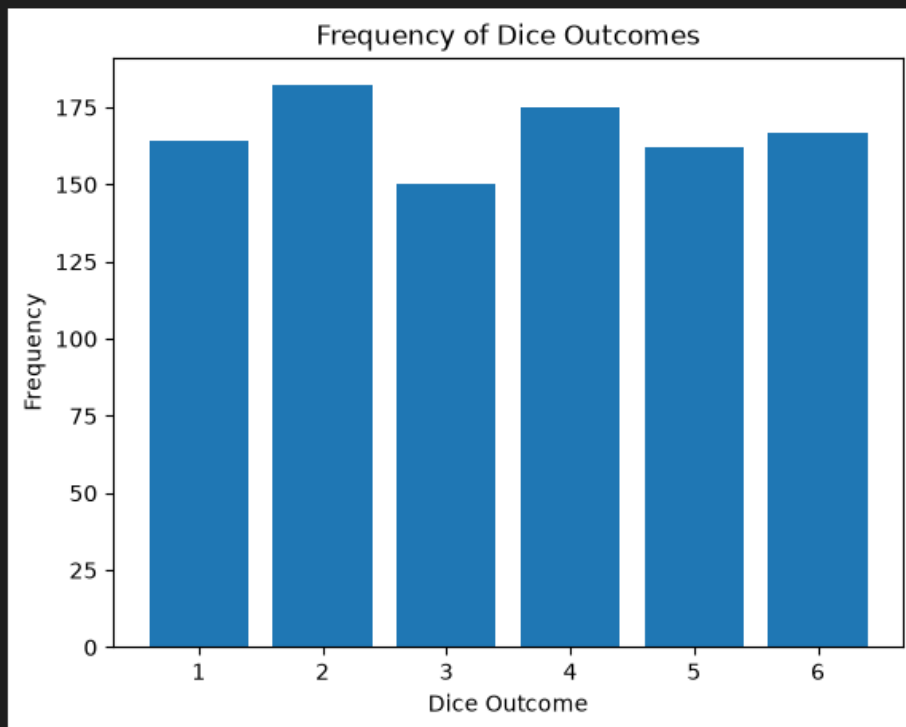
```

frequency = []
for i in range(1, 7):
    frequency.append(np.sum(dice_rolls == i))
print("Frequency:", frequency)
plt.bar(range(1, 7), frequency)
plt.xlabel("Dice Outcome")
plt.ylabel("Frequency")
plt.title("Frequency of Dice Outcomes")
plt.xticks(range(1, 7))
plt.show()

```

**Output:-**

```
Frequency: [np.int64(164), np.int64(182), np.int64(150), np.int64(175), np.int64(162), np.int64(167)]
```



**11. Create a bar chart showing the experimental probability of outcomes 1 to 6.**

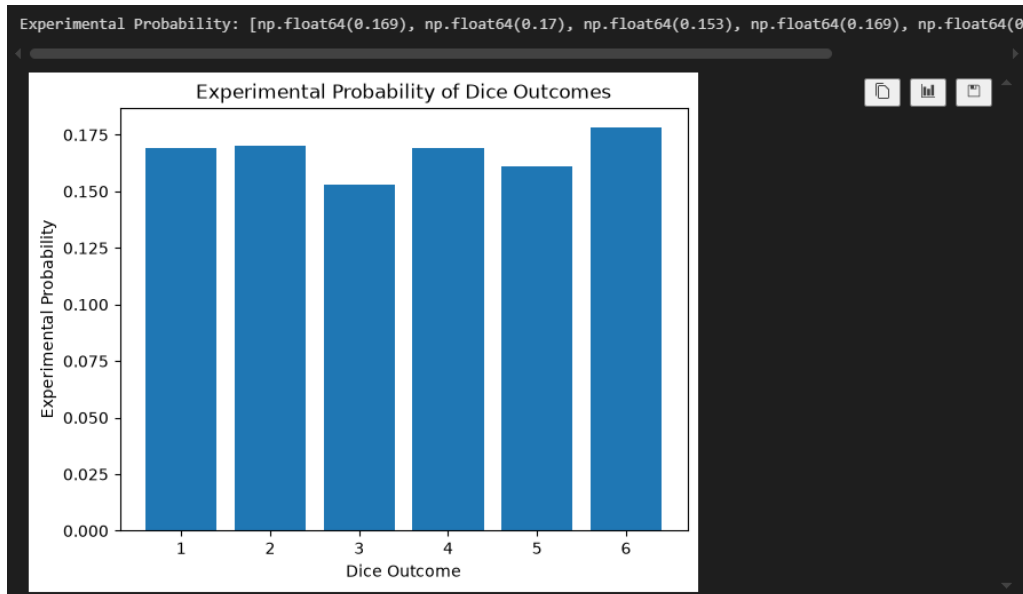
**Code:-**

```

import numpy as np
import matplotlib.pyplot as plt
dice_rolls = np.random.randint(1, 7, 1000)
probability = []
for i in range(1, 7):
    count = np.sum(dice_rolls == i)
    probability.append(count / 1000)
print("Experimental Probability:", probability)
plt.bar(range(1, 7), probability)
plt.xlabel("Dice Outcome")
plt.ylabel("Experimental Probability")
plt.title("Experimental Probability of Dice Outcomes")
plt.xticks(range(1, 7))
plt.show()

```

## Output:-

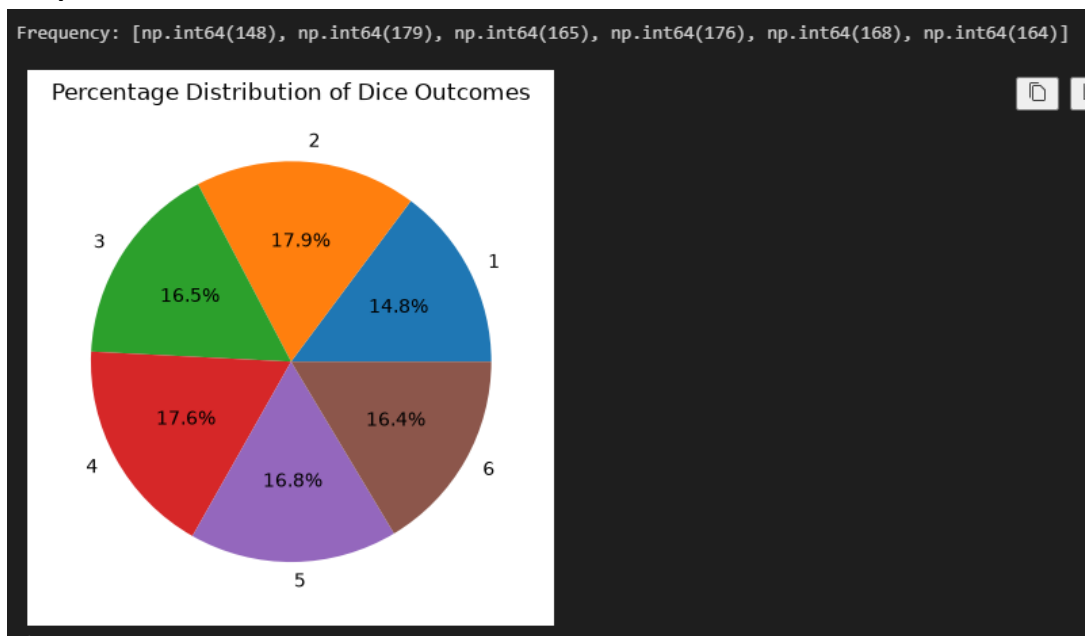


12. Create a pie chart showing the percentage distribution of dice outcomes.

### Code:-

```
import numpy as np
import matplotlib.pyplot as plt
dice_rolls = np.random.randint(1, 7, 1000)
frequency = []
for i in range(1, 7):
    frequency.append(np.sum(dice_rolls == i))
print("Frequency:", frequency)
plt.pie(
    frequency,
    labels=[1, 2, 3, 4, 5, 6],
    autopct="%1.1f%%"
)
plt.title("Percentage Distribution of Dice Outcomes")
plt.show()
```

## Output:-

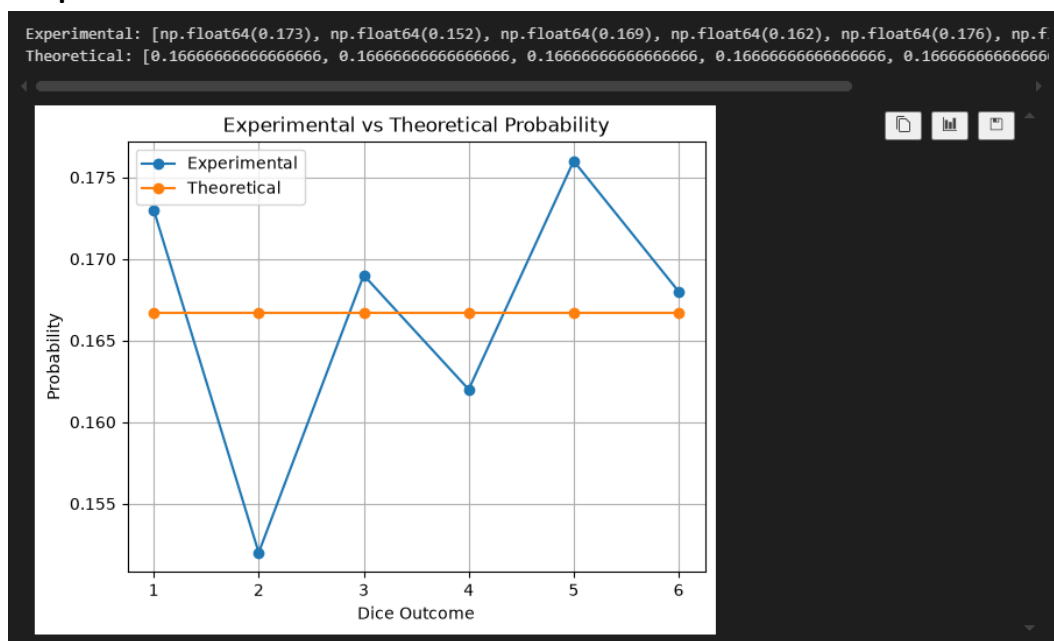


### 13. Create a line chart comparing experimental probability and theoretical probability for outcomes 1 to 6.

#### Code:-

```
import numpy as np
import matplotlib.pyplot as plt
dice_rolls = np.random.randint(1, 7, 1000)
experimental = []
for i in range(1, 7):
    experimental.append(np.sum(dice_rolls == i) / 1000)
theoretical = [1/6] * 6
print("Experimental:", experimental)
print("Theoretical:", theoretical)
plt.plot(
    range(1, 7),
    experimental,
    marker="o",
    label="Experimental"
)
plt.plot(
    range(1, 7),
    theoretical,
    marker="o",
    label="Theoretical"
)
plt.xlabel("Dice Outcome")
plt.ylabel("Probability")
plt.title("Experimental vs Theoretical Probability")
plt.legend()
plt.grid()
plt.show()
```

#### Output:-

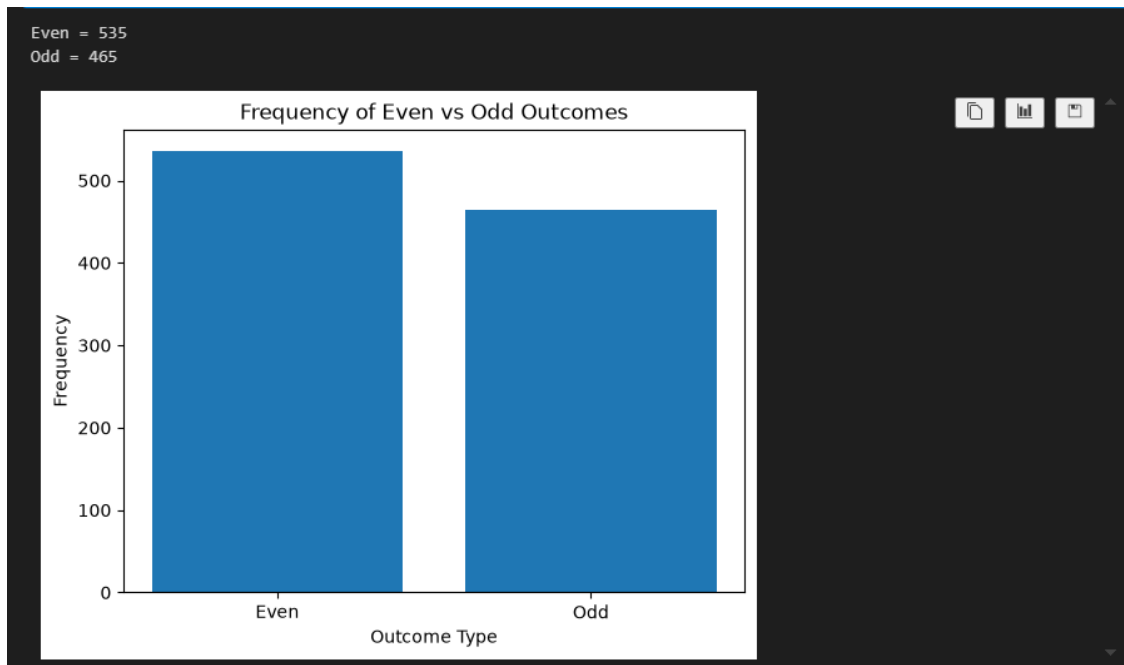


#### 14. Create a chart showing the frequency of Even vs Odd outcomes.

##### Code:-

```
import numpy as np
import matplotlib.pyplot as plt
dice_rolls = np.random.randint(1, 7, 1000)
even = np.sum(dice_rolls % 2 == 0)
odd = np.sum(dice_rolls % 2 != 0)
print("Even =", even)
print("Odd =", odd)
plt.bar(["Even", "Odd"], [even, odd])
plt.xlabel("Outcome Type")
plt.ylabel("Frequency")
plt.title("Frequency of Even vs Odd Outcomes")
plt.show()
```

##### Output:-



#### 15. Create a chart showing the experimental probability of:

- Even numbers
- Odd numbers
- Numbers greater than 4
- Numbers less than 4

##### Code:-

```
import numpy as np
import matplotlib.pyplot as plt
dice_rolls = np.random.randint(1, 7, 1000)
even = np.sum(dice_rolls % 2 == 0) / 1000
odd = np.sum(dice_rolls % 2 != 0) / 1000
greater_4 = np.sum(dice_rolls > 4) / 1000
less_4 = np.sum(dice_rolls < 4) / 1000
categories = [
    "Even",
```

```

"Odd",
"Greater than 4",
"Less than 4"
]
probabilities = [
    even,
    odd,
    greater_4,
    less_4
]
print("Experimental Probabilities:")
for i in range(len(categories)):
    print(categories[i], "=", probabilities[i])
plt.bar(categories, probabilities)
plt.xlabel("Event")
plt.ylabel("Experimental Probability")
plt.title("Experimental Probability of Different Events")
plt.xticks(rotation=15)
plt.show()

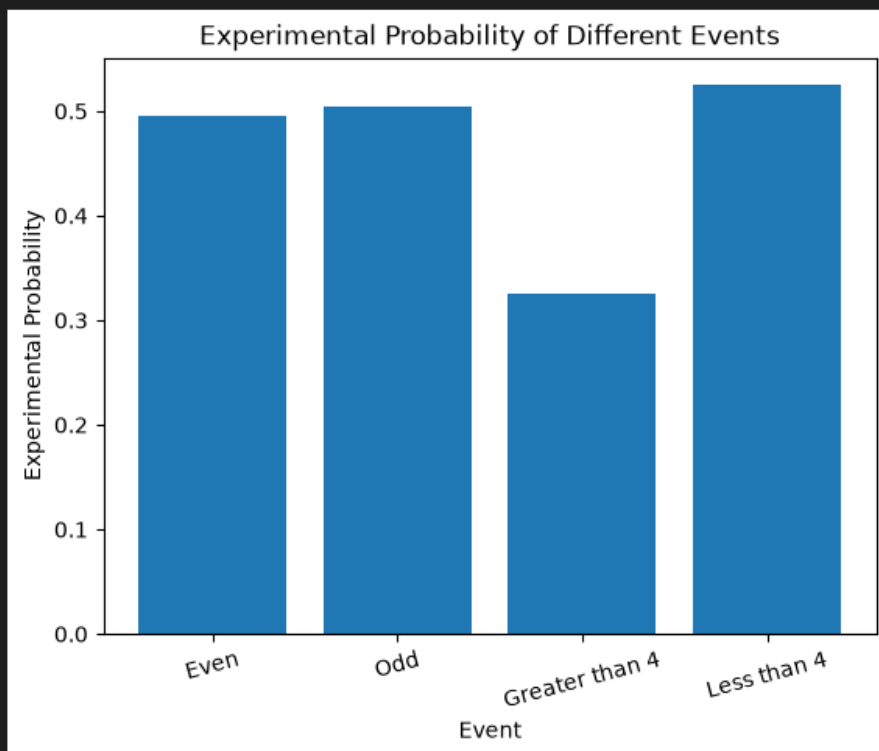
```

#### Output:-

```

Experimental Probabilities:
Even = 0.495
Odd = 0.505
Greater than 4 = 0.325
Less than 4 = 0.525

```



**b. Generate random numbers from various probability distributions (normal, uniform, exponential) and analyze their properties.**

**1. Generate 1,000 random numbers from a normal distribution with:**

**Mean = 50 and Standard deviation = 10**

**Code:-**

```
import numpy as np
normal_data = np.random.normal(50, 10, 1000)
print("First 20 values:")
print(normal_data[:20])
```

**Output:-**

```
First 20 values:
[56.41879061 46.16905079 40.76927604 47.30506244 49.23721848 33.38419011
 58.60332559 22.32674577 77.14816939 65.01879134 47.38991276 50.09426361
 53.02971874 44.10839612 51.92619148 48.56000022 49.05727283 51.81461063
 36.86472202 50.39569724]
```

**2. Calculate the mean, median, mode, variance, and standard deviation of the generated data.**

**Code:-**

```
import numpy as np
from scipy import stats
normal_data = np.random.normal(50, 10, 1000)
print("Mean =", np.mean(normal_data))
print("Median =", np.median(normal_data))
print("Mode =", stats.mode(normal_data, keepdims=True).mode[0])
print("Variance =", np.var(normal_data))
print("Standard Deviation =", np.std(normal_data))
```

**Output:-**

```
Mean = 50.206494346387395
Median = 50.07525837833538
Mode = 22.170395676567797
Variance = 99.05726039543981
Standard Deviation = 9.95275139825364
```

**3. Calculate the following statistics for the generated Normal distribution:**

- Mean
- Median
- Variance
- Standard deviation
- Minimum
- Maximum

**Code:-**

```
import numpy as np
normal_data = np.random.normal(50, 10, 1000)
print("Mean =", np.mean(normal_data))
print("Median =", np.median(normal_data))
print("Variance =", np.var(normal_data))
```

```
print("Standard Deviation =", np.std(normal_data))
print("Minimum =", np.min(normal_data))
print("Maximum =", np.max(normal_data))
```

**Output:-**

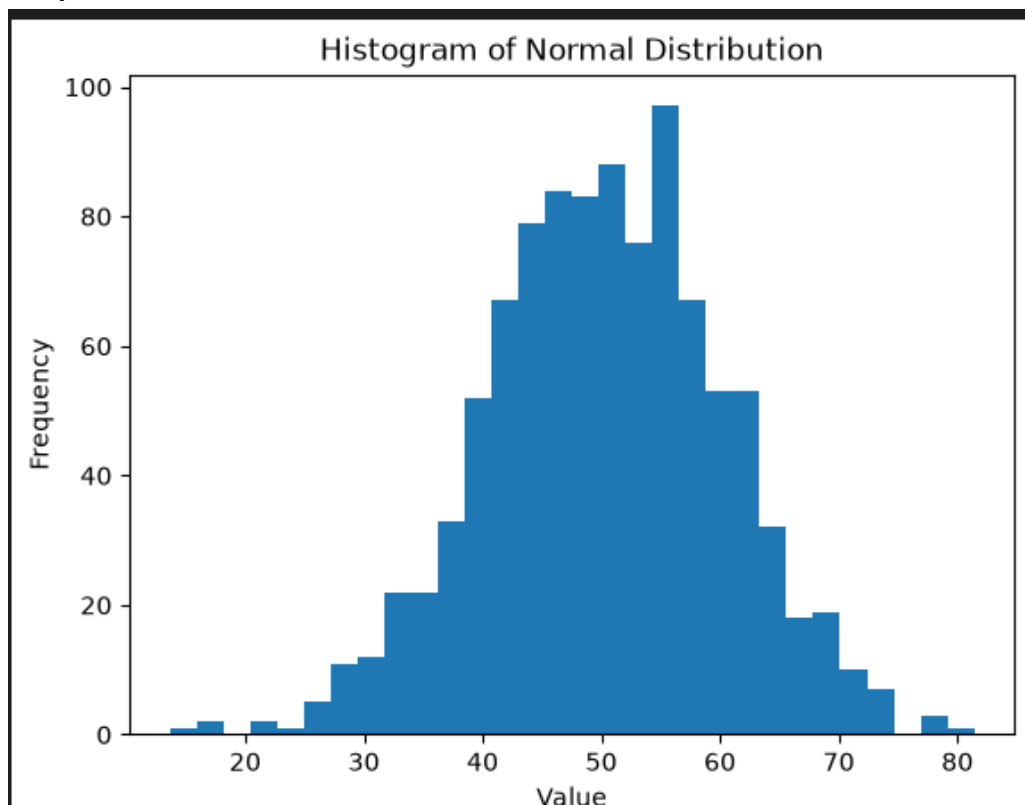
```
Mean = 50.29885246748564
Median = 50.3737956143769
Variance = 105.78363352243788
Standard Deviation = 10.285117088416538
Minimum = 19.174378248567624
Maximum = 84.28910478008214
```

**4. Create a histogram of the Normal distribution and analyze its shape.**

**Code:-**

```
import numpy as np
import matplotlib.pyplot as plt
normal_data = np.random.normal(50, 10, 1000)
plt.hist(normal_data, bins=30)
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Histogram of Normal Distribution")
plt.show()
```

**Output:-**



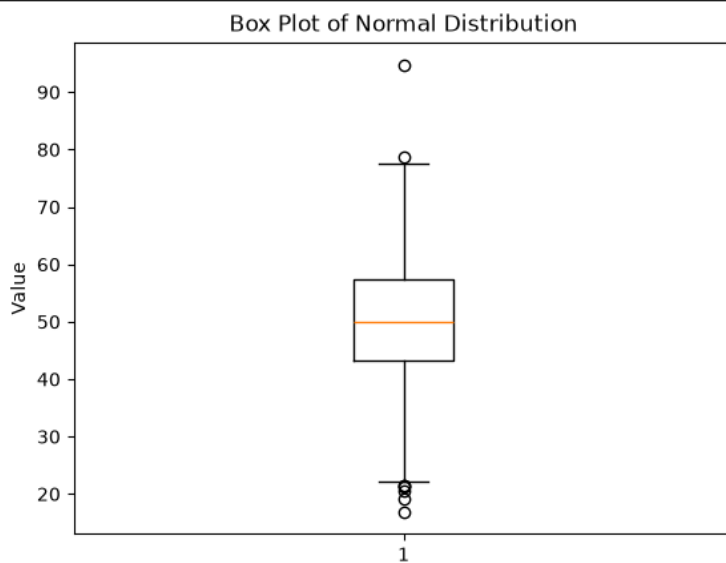
**5. Create a box plot of the Normal distribution.**

**Code:-**

```
import numpy as np
import matplotlib.pyplot as plt
normal_data = np.random.normal(50, 10, 1000)
plt.boxplot(normal_data)
```

```
plt.ylabel("Value")
plt.title("Box Plot of Normal Distribution")
plt.show()
```

**Output:-**



**6. Calculate the percentage of values that fall between:**  
 **$40 \leq X \leq 60$**

**Code:-**

```
import numpy as np
normal_data = np.random.normal(50, 10, 1000)
count = np.sum((normal_data >= 40) & (normal_data <= 60))
percentage = count / 1000 * 100
print("Number of values between 40 and 60 =", count)
print("Percentage =", percentage, "%")
```

**Output:-**

```
Number of values between 40 and 60 = 691
Percentage = 69.1 %
```

**7. Calculate the percentage of values that fall between:  $30 \leq X \leq 70$**

**Code:-**

```
import numpy as np
normal_data = np.random.normal(50, 10, 1000)
count = np.sum((normal_data >= 30) & (normal_data <= 70))
percentage = count / 1000 * 100
print("Number of values between 30 and 70 =", count)
print("Percentage =", percentage, "%")
```

**Output:-**

```
Number of values between 30 and 70 = 957
Percentage = 95.7 %
```

**8. Generate 1,000 random numbers from a uniform distribution between 0 and 100.**

**Code:-**

```
import numpy as np
```

```
uniform_data = np.random.uniform(0, 100, 1000)
print("First 20 values:")
print(uniform_data[:20])
```

**Output:-**

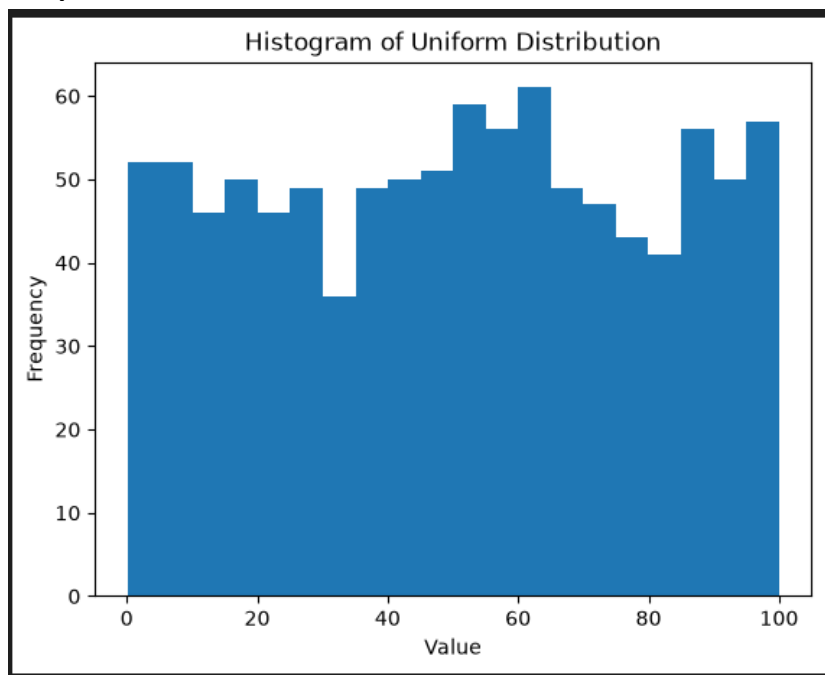
```
First 20 values:
[93.92477229 31.60797187 80.06914636 71.53470213 79.09176003 93.67681334
 62.33707602 89.79503434 16.76973229 62.6712576 77.22756129 27.16411307
 74.07538013 77.84768291 83.1187956 31.28829126 68.16065902 30.42532834
 73.90403875 46.29736651]
```

**9. Create a histogram of the Uniform distribution.**

**Code:-**

```
import numpy as np
import matplotlib.pyplot as plt
uniform_data = np.random.uniform(0, 100, 1000)
plt.hist(uniform_data, bins=20)
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Histogram of Uniform Distribution")
plt.show()
```

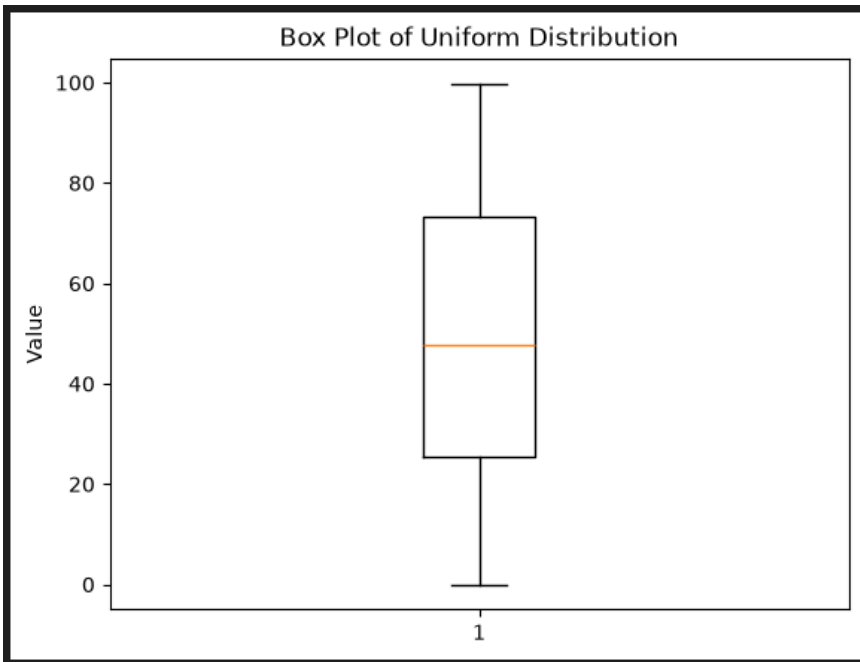
**Output:-**



**10. Create a box plot of the Uniform distribution.**

**Code:-**

```
import numpy as np
import matplotlib.pyplot as plt
uniform_data = np.random.uniform(0, 100, 1000)
plt.boxplot(uniform_data)
plt.ylabel("Value")
plt.title("Box Plot of Uniform Distribution")
plt.show()
```

**Output:-**

**11. Generate 1,000 random numbers from an Exponential distribution with scale = 2.**

**Code:-**

```
import numpy as np
exponential_data = np.random.exponential(2, 1000)
print("First 20 values:")
print(exponential_data[:20])
```

**Output:-**

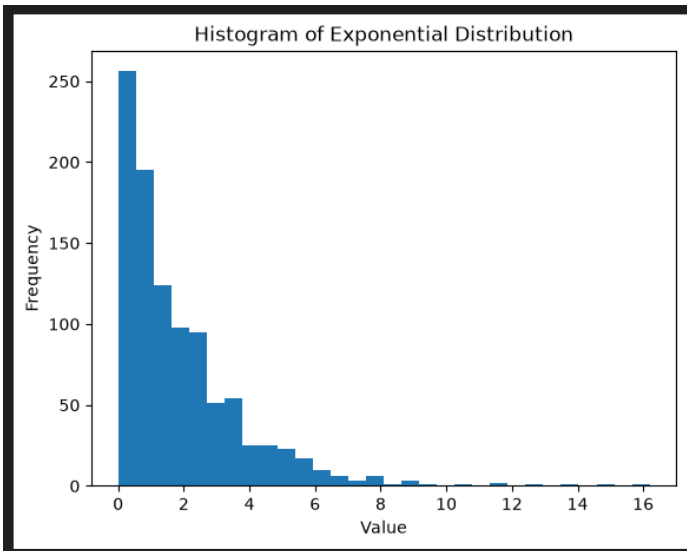
```
First 20 values:
[1.57171781 0.88497578 2.34401412 1.27525974 0.79815496 0.6161462
 5.19797568 0.68776993 1.14679708 0.86261658 0.18024076 0.41142864
 0.42021275 2.36605091 1.04372397 3.04622287 2.46852676 0.42650963
 1.65526398 2.71664914]
```

**12. Create a histogram of the Exponential distribution.**

**Code:-**

```
import numpy as np
import matplotlib.pyplot as plt
exponential_data = np.random.exponential(2, 1000)
plt.hist(exponential_data, bins=30)
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Histogram of Exponential Distribution")
plt.show()
```

**Output:-**

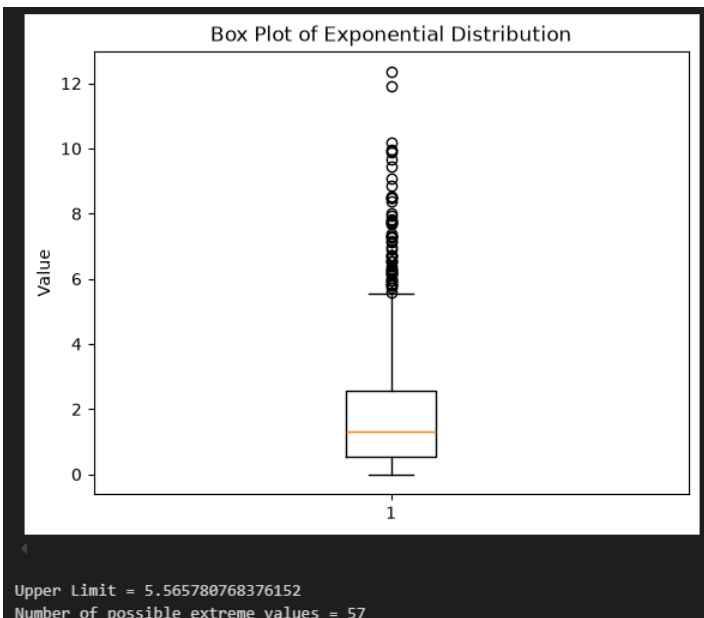


**13. Create a box plot and identify possible extreme values.**

**Code:-**

```
import numpy as np
import matplotlib.pyplot as plt
exponential_data = np.random.exponential(2, 1000)
plt.boxplot(exponential_data)
plt.ylabel("Value")
plt.title("Box Plot of Exponential Distribution")
plt.show()
Q1 = np.percentile(exponential_data, 25)
Q3 = np.percentile(exponential_data, 75)
IQR = Q3 - Q1
upper_limit = Q3 + 1.5 * IQR
extreme_values = exponential_data[
    exponential_data > upper_limit
]
print("Upper Limit =", upper_limit)
print("Number of possible extreme values =", len(extreme_values))
```

**Output:-**



#### 14. Calculate the percentage of values greater than 2.

##### Code:-

```
import numpy as np
exponential_data = np.random.exponential(2, 1000)
count = np.sum(exponential_data > 2)
percentage = count / 1000 * 100
print("Number of values greater than 2 =", count)
print("Percentage =", percentage, "%")
```

##### Output:-

```
Number of values greater than 2 = 356
Percentage = 35.6 %
```

#### 15. Calculate the percentage of values greater than 5.

##### Code:-

```
import numpy as np
exponential_data = np.random.exponential(2, 1000)
count = np.sum(exponential_data > 5)
percentage = count / 1000 * 100
print("Number of values greater than 5 =", count)
print("Percentage =", percentage, "%")
```

##### Output:-

```
Number of values greater than 5 = 86
Percentage = 8.6 %
```

#### C) Coin toss task (Include Prints)

##### 1. Simulate 1,000 coin tosses and calculate the experimental probability of:

- Getting Heads
- Getting Tails

##### Code:-

```
import numpy as np
coin_tosses = np.random.choice(
    ["Heads", "Tails"],
    1000
)
heads = np.sum(coin_tosses == "Heads")
tails = np.sum(coin_tosses == "Tails")
print("Total Tosses =", 1000)
print("Heads =", heads)
print("Tails =", tails)
print("Experimental Probability of Heads =", heads / 1000)
print("Experimental Probability of Tails =", tails / 1000)
```

##### Output:-

```
Total Tosses = 1000
Heads = 476
Tails = 524
Experimental Probability of Heads = 0.476
Experimental Probability of Tails = 0.524
```

## 2. Calculate the theoretical probability of Heads and Tails.

### Code:-

```
print("Theoretical Probability of Heads =", 1/2)
print("Theoretical Probability of Tails =", 1/2)
```

### Output:-

```
Theoretical Probability of Heads = 0.5
Theoretical Probability of Tails = 0.5
```

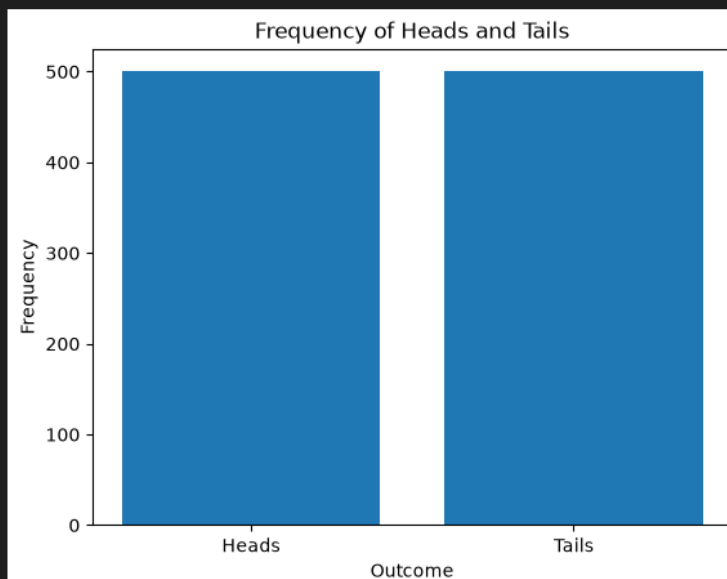
## 3. Create a bar chart showing the frequency of Heads and Tails.

### Code:-

```
import numpy as np
import matplotlib.pyplot as plt
coin_tosses = np.random.choice(
    ["Heads", "Tails"],
    1000
)
heads = np.sum(coin_tosses == "Heads")
tails = np.sum(coin_tosses == "Tails")
print("Heads =", heads)
print("Tails =", tails)
plt.bar(["Heads", "Tails"], [heads, tails])
plt.xlabel("Outcome")
plt.ylabel("Frequency")
plt.title("Frequency of Heads and Tails")
plt.show()
```

### Output:-

```
Heads = 500
Tails = 500
```



## 4. Create a pie chart showing the percentage of Heads and Tails.

### Code:-

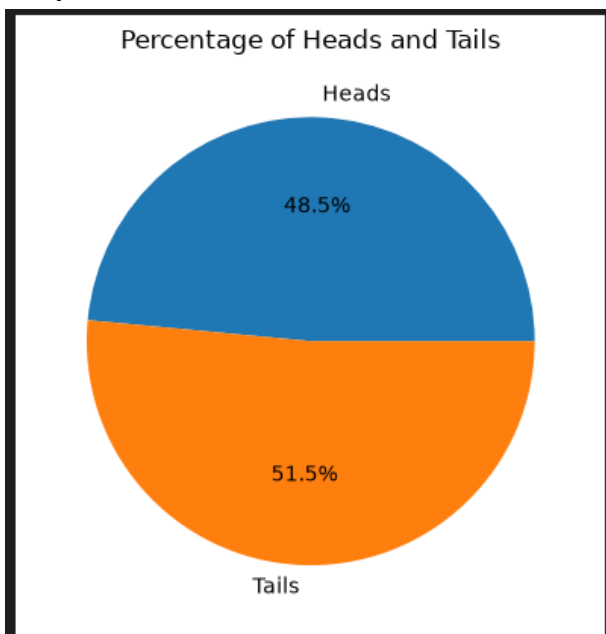
```
import numpy as np
import matplotlib.pyplot as plt
```

```

coin_tosses = np.random.choice(
    ["Heads", "Tails"],
    1000
)
heads = np.sum(coin_tosses == "Heads")
tails = np.sum(coin_tosses == "Tails")
plt.pie(
    [heads, tails],
    labels=["Heads", "Tails"],
    autopct="%1.1f%%"
)
plt.title("Percentage of Heads and Tails")
plt.show()

```

**Output:-**



**5. If a coin is tossed 10 times and Heads occurs 8 times, calculate the experimental probability of Heads.**

**Code:-**

```

heads = 8
total_tosses = 10
experimental_probability = heads / total_tosses
print("Total Tosses =", total_tosses)
print("Heads =", heads)
print("Experimental Probability of Heads =",
      experimental_probability)

```

**Output:-**

```

Total Tosses = 10
Heads = 8
Experimental Probability of Heads = 0.8

```

**6. Simulate 10,000 coin tosses and create a complete analysis containing:**

- Total tosses
- Number of Heads

- Number of Tails
- Experimental probability of Heads
- Experimental probability of Tails
- Theoretical probabilities
- Difference between experimental and theoretical probabilities
- Bar chart
- Pie chart

**Code:-**

```
import numpy as np
import matplotlib.pyplot as plt
# Simulate 10,000 tosses
coin_tosses = np.random.choice(
    ["Heads", "Tails"],
    10000
)
# Count Heads and Tails
heads = np.sum(coin_tosses == "Heads")
tails = np.sum(coin_tosses == "Tails")
# Experimental probability
experimental_heads = heads / 10000
experimental_tails = tails / 10000
# Theoretical probability
theoretical_heads = 0.5
theoretical_tails = 0.5
# Difference
difference_heads = abs(
    experimental_heads - theoretical_heads
)
difference_tails = abs(
    experimental_tails - theoretical_tails
)
# Print complete analysis
print("=====")
print("  10,000 COIN TOSS ANALYSIS")
print("=====")
print("Total Tosses =", 10000)
print("\nNumber of Heads =", heads)
print("Number of Tails =", tails)
print("\nExperimental Probability:")
print("Heads =", experimental_heads)
print("Tails =", experimental_tails)
print("\nTheoretical Probability:")
print("Heads =", theoretical_heads)
print("Tails =", theoretical_tails)
print("\nDifference:")
```

```

print("Heads =", difference_heads)
print("Tails =", difference_tails)
# Bar Chart
plt.figure(figsize=(7, 5))
plt.bar(
    ["Heads", "Tails"],
    [heads, tails]
)
plt.xlabel("Outcome")
plt.ylabel("Frequency")
plt.title("Heads vs Tails - 10,000 Tosses")
plt.show()
# Pie Chart
plt.figure(figsize=(7, 7))
plt.pie(
    [heads, tails],
    labels=["Heads", "Tails"],
    autopct="%1.1f%%"
)
plt.title("Percentage Distribution of Heads and Tails")
plt.show()

```

### Output:-

```

=====
      10,000 COIN TOSS ANALYSIS
=====
Total Tosses = 10000

Number of Heads = 4916
Number of Tails = 5084

Experimental Probability:
Heads = 0.4916
Tails = 0.5084

Theoretical Probability:
Heads = 0.5
Tails = 0.5

Difference:
Heads = 0.008400000000000019
Tails = 0.008399999999999963

```

